Barbell: An Extensible Generator of On-Demand Loads for Interactive Cloud Services

Yuxuan Liu[1 [0009−0001−4991−6031]], Yeh-Ching Chung[1 [0000−0002−8704−9821]], and Shuang Chen[2 [0000−0002−5392−1458]]

[1] School of Data Science, The Chinese University of Hong Kong, Shenzhen, China
yuxuanliu1@link.cuhk.edu.cn
ychung@cuhk.edu.cn
[2] Shuhai Lab, Huawei Cloud, Hangzhou, China
chenshuang44@huawei.com

Abstract. Interactive cloud service developers and infrastructure designers heavily rely on load generators to optimize service performance and infrastructure architecture. However, existing load generators are far from meeting their demands, for two reasons. First, they can only generate loads of peak or stable intensity in request rate, whereas our analysis of real-world traces shows that complicated load fluctuations are common and traces are often dumped in server resource utilization. Second, they are *tightly coupled* with their designated service type. This blocks the load generator development from keeping up with the needs of emerging cloud services such as large language model (LLM) inference.
To address these two issues, we propose an extensible load generator dubbed Barbell to generate load patterns on demand, with two innovations accordingly. First, we propose a multi-pass load intensity generation method to generate loads with a configurable pipeline of modular passes. A segment-wise aggregation pass is devised to generate loads with segments of distinct patterns by merging segment-specific passes. A profile-guided conversion pass is designed to convert loads in resource utilization to request rates. Second, Barbell designs a *decoupled* architecture where the load generation is decoupled from the service type with hook and callback interfaces. Four service extension models are proposed to extend to services with various types of clients and protocols.
Evaluation results show that Barbell achieves comparable accuracy of performance measures obtained by service type-specific load generators. It can generate loads that mimic real load patterns in the cloud. We have extended Barbell to six popular interactive services, including LLM inference, with only a few hundred lines of code for each service type.

Keywords: Load Generation · Workload Characterization · Performance Evaluation · Cloud Computing.

1 Introduction

Cloud data centers host a large number of interactive services such as web search and social media [14, 16]. Via these services, end users send requests to cloud

servers where the requests are processed and sent back to the users in a timely manner. Cloud servers experience significantly variant loads, depending on the number of end users, the request rate of each user, and the service type. Besides the existing ones, new interactive services such as large language model (LLM) inference [6] are *rapidly emerging* in the cloud, imposing new load patterns.

Load generators are widely used by service developers and infrastructure designers to test the performance of these services. They act as clients and emulate end user behaviors by generating synthetic loads [19]. It is critical to generate representative loads because the user heavily relies on them to understand and improve the performance of their services[15]. However, existing load generators are far from meeting their demands, specifically due to two challenges.

Challenge 1. Existing load generators are insufficient in emulating complicated but realistic load patterns. This makes them inappropriate to evaluate services hosted in real cloud. Most like sysbench[28] and memtier_benchmark[5] (memtier in short) can only generate peak loads or limit to a stable request rate. However, analysis of real cloud traces [18, 35] has shown that the services they work for barely operate at full or stable load. Mutilate[32] and OLTP-Bench[21] generate load variation with a single distribution or with step-wise fixed rates. Nevertheless, our analysis of CloudX traces shows that most periodic loads are diurnal, exhibiting distinct fluctuation patterns in different time periods. Facebook flashback[24] generates highly varying loads by replaying a given RPS trace, but fails when real traces are unavailable or when traces are not in request rate.

Challenge 2. Prior work also suffers from limited extensibility to various service types. Most existing load generators are designed to measure a specific service type with a specific load. For example, memtier [5] tests KV store services only, sysbench [28] tests relational database services only, and wrk [9, 10] tests web services only. Even for a single service, it has been proved that the incomplete support of request control policies [37] leads to limited extensibility and intolerable evaluation cost[27]. Take Memcached, for instance, it can be tested using memtier [5], mutilate [32], mutated [12], YCSB [17], etc. Memtier only supports stress testing; mutilate only supports closed-loop; while mutated and YCSB can operate in open-loop but not closed-loop. The performance measured by different policies are incomparable due to the coordination emission issue[39].

Our work. In this paper, we first analyze the limitations of prior work with concrete trace examples in Section 2. In Section 3, we extensively study load patterns of three diverse interactive services in CloudX, and find that unstable load is prevalent. Then, we propose Barbell in Section 4, which greatly improves the flexibility of load pattern generation and extensibility. We evaluate Barbell under various interactive cloud services to show its accuracy of measurement, flexibility of load pattern generation, and simplicity of extensibility in Section 5.

In summary, we make the following contributions:

1. We find that most cloud interactive services exhibit common load characterizations, calling for an extensible load generator to generate loads for them. 70% of VMs are unstable, but they exhibit common fluctuation patterns, including periodicity, diurnality, oscillation, and combinations of them.

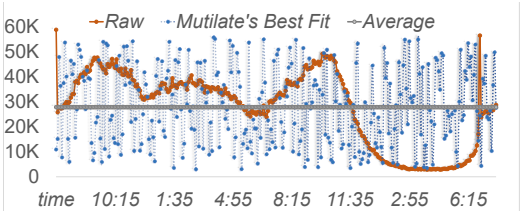


Fig. 1: A 24h RPS trace segment with diurnality in CloudX

2. We propose a multi-pass load intensity generation method to generate loads with a pipeline of passes that models elementary patterns, passes that aggregate them to generate combinatorial or load-segment-wise patterns, and passes that converts loads in resource utilization to RPS guided by profiling.
3. We design an extensible system architecture by decoupling general load generation functionality from service-specific ones. The general functionality has full support of request control policies and latency measurement, and we propose a methodology for each kind of interactive service to extend from it.
4. By putting it all together, we implement an extensible load generator named Barbell that can generate any desired load for any interactive cloud services.

2 Prior Work Limitation

2.1 Single-Pass Load Intensity Generation

Existing load generators fail to generate complicated load patterns because they model them in a single-pass way, trying to fit all patterns within an entire trace with a single model. The easiest model is to limit request rate to the peak or average. Many contemporary load generators, such as sysbench[28] and memtier[5], implement stress testing by issuing requests as fast as possible. Most of them can limit to a fixed rate, but cannot generate load fluctuation at all. To address this issue, mutilate[32] and OLTP-Bench[21] generates load variation with a single distribution or step-wise fixed rates. However, they fail to fit common load patterns like diurnality. Figure 1 shows a typical RPS trace segment with diurnality in CloudX. Mutilate's best-fitted distributions completely loses the diurnality.

Time series[22, 25, 41] models can fit complicated load variations, but the parameters (e.g., Hurst coefficient[36]) are obscure for users without expertise to tune. Besides, they require long (quadratic) time to generate large traces[40]. Facebook flashback[24] can model any load patterns by replaying a given RPS trace. Nevertheless, when the trace is not in RPS, the user has to do a manual transformation for every load on every server under test.

2.2 Coupled General and Service-Specific Functionality

Existing load generators have limited extensibility to service types because they tightly couple general functionality to service-specific ones. Most of them are

Table 1: Feature of VM's Load Pattern

Feature	Description
Stability	Range of the original series $< 5\%$
Periodicity	ACF of the denoised series > 0.5
Fluctuation	CV of the residual series $> 10\%$

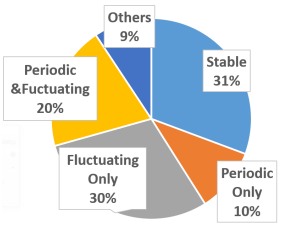


Fig. 2: Percentage of each type of VM.

designated for a specific service type with specific interfaces. For example, mutilate[32] only implements get/set for KV store services, and sysbench[28] only implements SQL queries like select/insert for relational database services. The tight coupling lowers the development cost, but leads to significant heterogeneity across programming languages (C/C++, Java, Go), configuration methods (CLI, XML, Lua), and request control policies (closed-loop, open-loop, and partly-open-loop[37]). The heterogeneity causes intolerable cost in deployment and configuration, and incomparable measurements for a comprehensive evaluation[27].

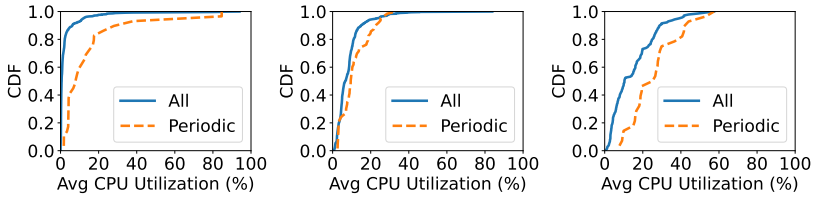
Tailbench[26] and lancet[27] have pioneered the design of extensible load generators for general interactive services, by forcibly serializing all requests into socket streams. It works well for services with simple and stateless protocols like Redis, but fails for services with complicated or stateful protocols like MySQL. Developers have to manage every protocol detail, and cannot use off-the-shelf clients and connectors. Sysbench does similar thing by manually defining all SQL data structures and member fields, but it takes hundreds to thousands LoC, and the developers have to continuously fix compatibility issues[**?**,**?**,**?**].

3 Workload Characterization

3.1 Dataset & Feature Extraction

Our dataset includes three thousand VMs and about six million data points from twenty tenants and three cloud interactive services hosted in CloudX, namely, Relational Database Service (RDS, such as MySQL), Distributed Caching Service (DCS, such as Memcache), and Distributed Message Service (DMS, such as Kafka). For each VM, we perform the following steps to extract its load pattern:

1. Calculate the minimum, mean, and maximum. This gives us a general view of how large the VM's load is, and how large the load variation range is.
2. Apply Autoregressive Integrated Moving Average (ARIMA) model [38] for time series analysis. Specifically, we decompose one load time series into three: trend, seasonality, and residuals. Combining trend and seasonality, we obtain the denoised and the residual series. The goal is to exclude the impact of residual when analyzing periodicity, and better understand fluctuations.



(a) DCS (b) RDS (c) DMS

Fig. 3: CDF of average CPU utilization of all and periodic VMs for three services.

3. Calculate the autocorrelation function (ACF) [3, 31] of the denoised series. ACF reflects how periodic the time series is. An absolute value closer to 1 represents stronger periodicity. We do not obtain ACF of the original load time series to exclude the impact of unpredictable spikes or fluctuations.
4. Divide the standard deviation of the residual series by the mean of the denoised series. Intuitively, this calculates the coefficient of variation (CV) [11] of the load and represents how fluctuated it is.
5. Categorize VMs based on the metrics calculated above, as shown in Table 1. The percentage of each type of VM is plotted in Figure 2).

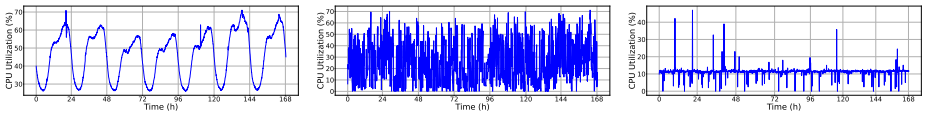
3.2 Load Characteristics in the Cloud

In summary, we observe two key characteristics: load intensity is mostly small to medium, and load variation shows periodic and fluctuating behavior.

Load Intensity Characteristics It is well known that resource utilization is low in the cloud [18, 23, 20, 33]. However, after categorizing VMs by their stability, we find that unstable VMs have much higher loads than stable VMs. Figure 3 shows the distribution of the average CPU utilization of all VMs and periodic VMs for each interactive service. The average of DCS, RDS, and DMS VMs is 2.7%, 8.4%, and 14.8%, respectively. DCS has the lowest average CPU utilization among them because it (i.e., KV store) is a memory-intensive service.

All stable VMs have an average CPU utilization under 10%. 64% of them even have loads lower than 3%, signaling that no workload is running inside. Since most cloud service providers provide monthly or yearly subscriptions cheaper than on-demand pricing [1, 2], many users almost never release their resources even when their VMs are completely idle. Contrarily, periodic VMs have fluctuating loads, meaning that they are actively in use and, therefore, have a higher load, as shown in Figure 3. CDF means cumulative distribution function.

Load Variation Characteristics Most periodic VMs have diurnal load patterns. Figure 4a is purely periodic with clear diurnality in a repeating period of 24 hours. Fluctuating VMs have various fluctuating patterns. Figure 4b shows an oscillating pattern, while Figure 4c is more bursty.



(a) Purely Periodic VM with Diurnal Loads (b) Fluctuating VM with Oscillating Load (c) Fluctuating VM with Bursty Load

Fig. 4: Load traces with different variation patterns in CloudX.

The best fitting distributions for unstable VMs are generalized normal distribution (gennorm), alpha distribution (alpha), and exponentially modified Gaussian distribution (exponorm).

4 Barbell Design

4.1 System Overview

Given the limitations of prior work (Section 2) and the load patterns in the real cloud (Section 3), we design Barbell, an extensible load generator to generate on-demand loads for interactive cloud services. Figure 5 shows the overall system design of Barbell, highlighting three key design choices:

- **Flexible load pattern generation** is realized through multi-pass load intensity generation with a custom pipeline of modular passes.
- **Request control policies are fully supported** in general functionality by managing the number of concurrent in-flight requests in each connection.
- **Extensibility** is realized through decoupling general and service-specific functionalities (extension in short) with the hook and callback interface.

4.2 Multi-Pass Load Intensity Generator

The core of multi-pass load intensity generator is a pipeline of multiple passes, where each pass scans the entire trace from the previous pass, performs some adjustments, and output a new trace to the next pass or directly to the load executor. All intermediate results (IR) are time series (not necessarily in RPS), so the passes are modular and pluggable. Users can easily develop new passes to identify desired load patterns and apply them in the custom pass pipeline.

Entrance Pass (Synthesis / Replay)
The pass pipeline starts with an entrance pass (either a synthesis pass or a replay pass) to generate traces with raw patterns for all following passes. The *synthesis* pass fills the trace with random intensity generated by the specified distribution. The *replay* pass fills the trace with data points of an input trace, which can be collected from real clouds, public datasets, or third-party trace fitters.

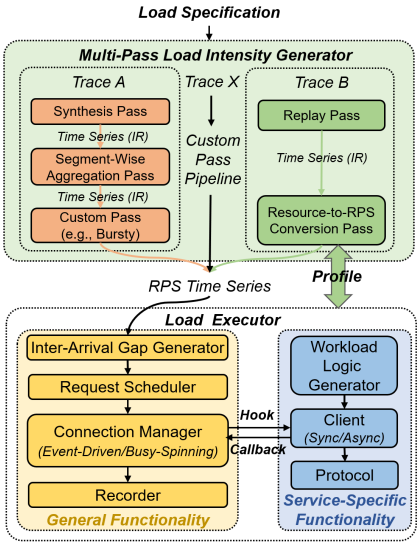


Fig. 5: System overview of Barbell.

Segment-wise Load Aggregation Pass
This pass aggregates input traces with distinct load patterns to an output trace of multiple segments. Each segment exhibits the load pattern of an input trace. Take diurnal loads in figure 1 for instance, it exhibits distinct patterns during daytime and nighttime, so the pass aggregates traces from two synthesis passes. These two passes are configured with best fitted distribution to retain load patterns of daytime and nighttime separately. Finally, the pass aggregates input traces into one according to the duration of each segment and output it.

Similar to segment-wise load aggregation, passes can be easily developed to generate other load patterns in a modular fashion. For example, there can be a pass to generate load periodicity, and another pass to generate bursty loads of specified interval and intensity upon input traces.

Profile-Guided Resource to RPS Load Conversion Pass
Unlike above passes that work with static configuration, converting traces from resource utilization to RPS requires approximating RPS through interpolation of dynamically profiled data on the server under test. Among all common resource utilization, Barbell implements the prevalent scaled CPU load, and others can be implemented similarly. The pass first determines several representative CPU loads as targets to profile. These targets are the samples for interpolation, and the accuracy of approximation increases with finer-grained target selection. Test runs are then launched to approach each target gradually.

The initial test run works as stress testing to get the first RPS-CPU mapping without prior knowledge of the server. The pass then iteratively generates loads

of fixed RPS, gets stabilized average CPU load, and update RPS for a new test run. A target is profiled successfully when a test run produces CPU load close enough to it within an error tolerance, which determines confidence of profiling. It is configurable and the default value is $\pm 2\%$ balancing both profiling time and accuracy. After all the targets are profiled successfully, the pass approximates the RPS of every CPU load in the input trace through piecewise linear interpolation. The profiling is efficient because its time complexity is linear to the range of scaled CPU load to profile, and is irrelevant to the length of the input sequence.

4.3 General Functionality

Request Scheduler

It manages a state machine for every request during load generation. A request initiates at the waiting state once the inter-arrival gap generator determines its arrival time. It then transits to the scheduled state when its arrival time arrives. There is a request queue in each connection to store in-flight requests that are sent out but have not received complete responses. Requests in the scheduled state sustain client-side waiting until an in-flight request completes.

The three request control policies are then controlled by the number of concurrent in-flight requests. Closed-loop policy allows only one global in-flight request among all connections. Open-loop policy supports unlimited concurrent in-flight requests, and the queue should have no size limit. Partly-open-loop policy lies in between. The larger the number of concurrent in-flight requests is, the more similar it behaves like open-loop, and vice versa.

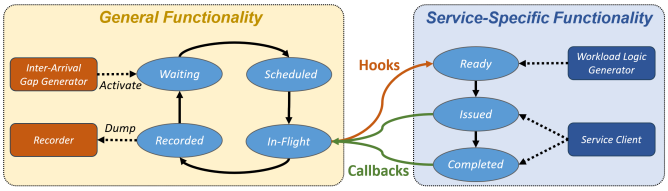


Fig. 6: State transition diagram of a request in Barbell.

Connection Manager

It ensures immediate issue of requests in the scheduled state whenever the request queue is not full to avoid latency underestimation due to coordination emission[39]. For closed-loop policy, it adopts the event-driven model to issue requests and receive responses in a single event loop. The interleaved execution produces accurate measurements because senders and receivers never overlap. When it comes to (partly-)open-loop policy, the event-driven model suffers from missed sends and receives because concurrent senders and receivers overlap. Barbell solves the issue by hosting the sender and receiver with separate threads. Both threads use busy-spinning to reduce client-side jitters when loads are heavy.

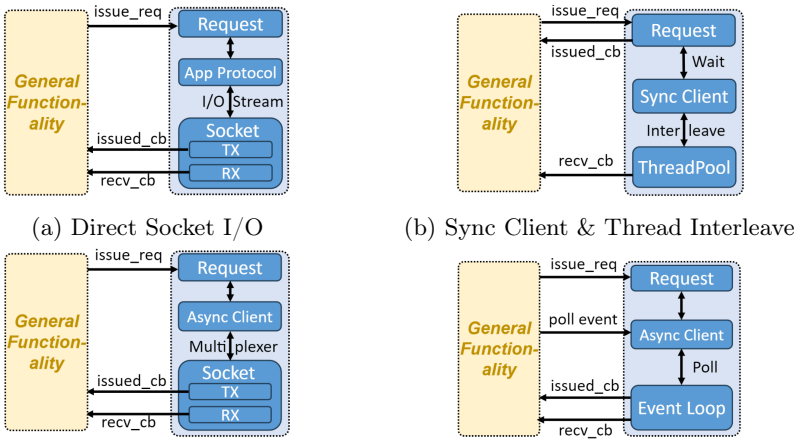
Hook and Callback Interface
Whenever a request enters the request queue, the connection manager notifies the service extension via *issue_ req* hook. The request first transits to the ready state when the workload logic generator determines its request body (more details in Section 4.4). Then, the service client issues the request to the server (the issued state) and receives the response (the completed state) later, triggering *issued_ cb* and *recv_ cb* callbacks to notify the connection manager to measure client-side waiting time, end-to-end latency, and service-specific statistics if attached.

4.4 Service-Specific Functionality

Models for Service Extension
We categorize service extensions as four models (see figure 7) for services with different types clients and protocols. Currently, the six popular interactive cloud services that Barbell implements demonstrate the usage of all four models.

- **Direct Socket I/O** [*Redis, Memcached*]. The extension issues requests as I/O streams through built-in socket (TCP/UDP), and the developers issues requests and parses responses as streams following the service protocol. The *issued_ cb* and *recv_ cb* are triggered when the streams are loaded to the write buffer, and responses arrive at the read buffer separately.
- **Sync Client with Thread Interleaving** [*MySQL*]. The extension issues requests through client APIs in a synchronous way. The client manages protocol details and issues multiple requests simultaneously by launching multiple interleaved threads as a thread pool. Each incoming request fits an idle thread, and the pool size limits the number of concurrent in-flight requests.
- **Async Client with Stateless Protocol** [*Nginx, vLLM*]. Requests can be issued in a mutually independent way. The client issues them asynchronously through I/O multiplexing, and its built-in callbacks are implicitly triggered when it finishes issuing a request and receiving the response.
- **Async Client with Stateful Protocol** [*Kafka (Producer and Consumer)*]. The client must manage session history, request ordering and execution status. The extension still issues requests asynchronously, but has to explicitly pool the client occasionally to activate the I/O event and callbacks.

Workload Logic Generator
It determines the number and type of requests to issue along with all associated arguments. For *request-based* loads, where requests/transactions are mutually independent, it only generates the request body (e.g., key and value streams for KV store and web URLs for HTTP). It takes service-specific configurations generate workload logic that adheres desired features (e.g., hot key space and temporal locality). Barbell also supports generating *session-based* loads, where inter-request dependency exists[13]. An example is YCSB's core workload F of read-modify-write transactions[17]. The user reads a key, and updates it with a modified value. Barbell allows users to write a Lua script to program the request



(a) Direct Socket I/O

(b) Sync Client & Thread Interleave

(c) Async Client & Stateless Protocol

(d) Async Client & Stateful Protocol

Fig. 7: Four models of service extension in Barbell.

flow, then it loads and executes the script during runtime. Sysbench proposed similar configuration method, but they only work for relational database service.

5 Evaluation

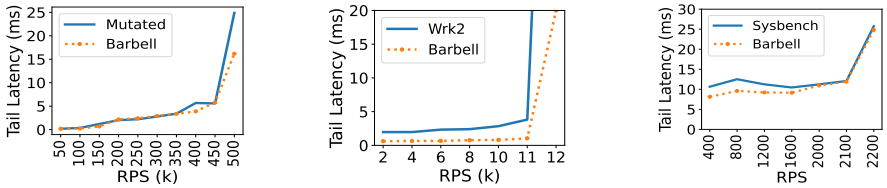
We evaluate Barbell's accuracy of measurement in Section 5.2, flexibility of load pattern generation in Section 5.3, and simplicity of extensibility in Section 5.4.

5.1 Methodology

We use two Intel Xeon Silver 4210 machines running Ubuntu 20.04, each with 20 physical cores (40 threads) in 2 sockets and 96GB memory. The machines have hyper-threading and turbo boost enabled. We launch the latest stable release of each load generator in a 16-core docker container on one machine as client and each service in a 4-core docker container on the other machine as server. There is no difference between VM and docker to host the service as long as the overall resource consumption is under proper constraint. All containers are pinned to a fixed set of physical cores in the same socket, and use `-network host` to ensure stable performance measurement. Table 2 shows the dataset and load generators for each test service. Unless otherwise specified, load generators and services are instantiated with 8 and 4 threads/workers, separately. Each test lasts for 120 seconds. 30B and 200B are average key/value sizes among CloudX's DCS loads.

Table 2: Service Throughput Comparison with SOTA Load Generators

Service	Dataset	Load Generator	Maximum RPS
Memcached	10k key range, SET:GET=1:9 <Key(30B), Value(200B)>	Mutilate v0.1	363452
		Barbell	368494 (+1.39%)
Nginx	GET method on files 10KB file size	wrk2 v4.0.0	11403
		Barbell	11396 (-0.06%)
MySQL	OLTP read-only transactions 4 tables, each with 10k entries	Sysbench v1.0.20	2345
		Barbell	2007 (-14.41%)



(a) Memcached (b) Nginx (c) MySQL

Fig. 8: 99th percentile of latency (tail latency) with increasing RPS.

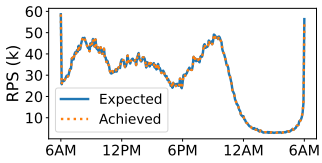


Fig. 9: The replay pass precisely replays trace in Figure 1.

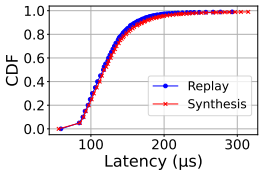


Fig. 10: Latency CDF under replay and synthesis pass pipeline.

5.2 Accuracy of Measurements

We evaluate Barbell's accuracy in throughput and latency measurement and show that Barbell can achieve comparable capability with other service-specific load generators that have limited request control policy support.

The maximum throughput under closed-loop policy reflects load generators' ability for typical stress testing. As Table 2 shows, Barbell achieves similar throughput with SOTA service-specific load generators for Memcached and Nginx with difference less than 2%. Compared with sysbench, Barbell underperforms by 14% in MySQL transaction throughput because Barbell implements the extension with off-the-shelf C++ connectors[8] and high-level LuaBridge[34] for Lua binding. It trades off some performance loss for considerably greater robustness due to complete encapsulation of protocol details.

Table 3: Decomposition of Barbell's LoC. 73% of LoC is in general functionality.

Functionality	General	Service Extension					
		Memcached	Redis [7]	Nginx	vLLM [30]	Kafka [29]	MySQL
LoC	6,431	579	566	229	246	496	284

Latency under open-loop policy reflects load generators' ability of faithful latency measurement by efficiently managing concurrent in-flight requests with minimal client-side jitters. Figure 8 shows that tail latency measured by Barbell shows the same trend to other load generators with increasing RPS. Latency remains stable at low load and increases dramatically after a specific RPS because of exponentially growing server-side queuing delay. Generally, Barbell reports tail latency similar to or closer to that measured by others before and after reaching the maximum closed-loop throughput.

5.3 Flexibility of Load Pattern Generation

We prove it by using two pass pipelines to generate trace in figure 1. The first pipeline consists of a single replay pass, and the other consists of a synthesis pass and a segment-wise load aggregation pass. The effectiveness of the Resource-to-RPS load conversion is trivial because it is guided by server-specific profiling.

Barbell can generate fine-grained fluctuating loads. Figure 9 shows the expected RPS (i.e., the input RPS) and the achieved RPS (i.e., the RPS that the server actually receives). It is clear that Barbell faithfully replays the trace; R^2 [4] of the achieved RPS is 0.99995 (the closer to 1, the better).

Barbell can generate complicated load patterns in real traces. The synthesis pass generates loads of each segment with best-fitted `gennorm` and `alpha` distributions. The Kullback-Leibler (KL) divergence reduces to 0.049 (smaller the better) and 0.171 for daytime and nighttime, compared to 0.475 by fitting the entire trace. Although it cannot retain every rise and fall, Figure 10 shows that latency CDF is almost the same to that of trace replaying.

5.4 Simplicity of Extensibility

Barbell is extensible to various interactive cloud services, including traditional ones running on CPUs and emerging ones running on GPUs and accelerators like LLM inference. The majority of codes (73%) are spent enriching the reusable general functionality, such as complete support of request control policies and latency measurement. Extending to a new service only takes 200-600 LoC (Table 3), which is much less than developing a new load generator from scratch.

Both service-agnostic (Server IP & port, etc) and service-specific attributes are configured via one YAML file. Figure 11 shows the vLLM-specific fields in the YAML file, including the path to the dataset, the interested range of prompt and decode lengths, and the URL to the vLLM API server.

```yaml
RequestContentCustomization:
  vLLM:
    DataSetPath: /root/work/dataset/Qwen2-0.5B_dataset.json
    PromptLengthMax: 1024
    PromptLengthMin: 1024
    DecodeLengthMax: 32
    DecodeLengthMin: 32
    RandomSeed: 33
    UrlPath: http://150.1.68.169:8080/generate
```

Fig. 11: vLLM-specific configurations in YAML.

6 Conclusion & Future Work

We presented Barbell, a multi-threaded extensible load generator for interactive cloud services. It can generate loads with complicated but real patterns to emulate end user behavior through its multi-pass load intensity generation method. It fully decouples general and service-specific functionalities. New services can be easily extended with its powerful and complete general functionality.

Future work will focus on workload logic characterization and improving Barbell's ability to generate OLAP workloads with complex analytical logic.

Availability

To help further research on fluctuating load in the cloud, We open-source some load traces in CloudX, including one-week CPU utilization time series for 20 selected representative VMs. For Barbell, we prepare a docker image open for download and trial. Please visit `https://anonymous.4open.science/r/Barbell-39CE`.

Acknowledgment

We sincerely thank Zhibin Yu, Lv Chen and Zhangcheng Liang from Huawei Cloud for their extensive feedback on this work. This work was supported in part by an industry-university collaboration research project between Huawei Cloud and CUHK (Shenzhen).

Statements on AI Use

The technical solution and experimental design presented in this paper were independently completed by the authors. AI tools were used solely for language polishing and formatting optimization, and did not participate in the development of the research ideas or core content.

References

1. Alibaba cloud elastic compute service. `{https://www.alibabacloud.com/product/ecs}`

2. Amazon ec2. {https://aws.amazon.com/ec2/}
3. class statsmodels.tsa.stattools.acf. {https://www.statsmodels.org/stable/generated/statsmodels.tsa.stattools.acf.html}
4. Coefficient of determination. {https://en.wikipedia.org/wiki/Coefficient_of_determination}
5. Memtier_benchmark. https://github.com/RedisLabs/memtier_benchmark
6. Openai api. {https://platform.openai.com/}
7. Redis: The real-time data platform. {https://redis.io}
8. Mysqlconnectorcpp. https://github.com/mysql/mysql-connector-cpp (2008)
9. Wrk. https://github.com/wg/wrk (2015)
10. Wrk2. https://github.com/giltene/wrk2 (2015)
11. Abdi, H.: Coefficient of variation. Encyclopedia of research design **1**(5), 169–171 (2010)
12. Adam Belay, David Terei, E.Z.Y.: Mutated. https://github.com/scslab/mutated (2015)
13. Bahga, A., Madisetti, V.K., et al.: Synthetic workload generation for cloud computing applications. Journal of Software Engineering and Applications **4**(07), 396 (2011)
14. Barroso, L.A., Clidaras, J., Hölzle, U.: The datacenter as a computer: An introduction to the design of warehouse-scale machines. Synthesis lectures on computer architecture **8**(3), 1–154 (2013)
15. Beitch, A., Liu, B., Yung, T., Griffith, R., Fox, A., Patterson, D.A., et al.: Rain: A workload generation toolkit for cloud computing applications. UC Berkeley Technical Publications (UCB/EECS-2010-14) (2010)
16. Chen, S., Delimitrou, C., Martínez, J.F.: Parties: Qos-aware resource partitioning for multiple interactive services. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 107–120 (2019)
17. Cooper, B.F., Silberstein, A., Tam, E., Ramakrishnan, R., Sears, R.: Benchmarking cloud serving systems with ycsb. In: Proceedings of the 1st ACM symposium on Cloud computing. pp. 143–154 (2010)
18. Cortez, E., Bonde, A., Muzio, A., Russinovich, M., Fontoura, M., Bianchini, R.: Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In: Proceedings of the 26th Symposium on Operating Systems Principles. pp. 153–167 (2017)
19. Curiel, M., Pont, A.: Workload generators for web-based systems: Characteristics, current status, and challenges. IEEE Communications Surveys & Tutorials **20**(2), 1526–1546 (2018)
20. Delimitrou, C., Kozyrakis, C.: Quasar: Resource-efficient and qos-aware cluster management. ACM SIGPLAN Notices **49**(4), 127–144 (2014)
21. Difallah, D.E., Pavlo, A., Curino, C., Cudre-Mauroux, P.: Oltp-bench: An extensible testbed for benchmarking relational databases. Proceedings of the VLDB Endowment **7**(4), 277–288 (2013)
22. Feitelson, D.G.: Workload modeling for performance evaluation. In: IFIP International Symposium on Computer Performance Modeling, Measurement and Evaluation. pp. 114–141. Springer (2002)
23. Guo, J., Chang, Z., Wang, S., Ding, H., Feng, Y., Mao, L., Bao, Y.: Who limits the resource efficiency of my datacenter: An analysis of alibaba datacenter traces. In: Proceedings of the International Symposium on Quality of Service. pp. 1–10 (2019)

24. Inc., F.: Flashback. https://github.com/facebookarchive/flashback (2015)
25. Kant, K., Tewari, V., Iyer, R.K.: Geist: A web traffic generation tool. In: Proceedings of the 12th International Conference on Computer Performance Evaluation, Modelling Techniques and Tools. p. 227–232. TOOLS '02, Springer-Verlag, Berlin, Heidelberg (2002)
26. Kasture, H., Sanchez, D.: Tailbench: a benchmark suite and evaluation methodology for latency-critical applications. In: 2016 IEEE International Symposium on Workload Characterization (IISWC). pp. 1–10 (2016). https://doi.org/10.1109/IISWC.2016.7581261
27. Kogias, M., Mallon, S., Bugnion, E.: Lancet: A self-correcting latency measuring tool. In: 2019 USENIX Annual Technical Conference (USENIX ATC 19). pp. 881–896. USENIX Association, Renton, WA (2019), https://www.usenix.org/conference/atc19/presentation/kogias-lancet
28. Kopytov, A.: Sysbench manual. MySQL AB pp. 2–3 (2012)
29. Kreps, J., Narkhede, N., Rao, J., et al.: Kafka: A distributed messaging system for log processing. In: Proceedings of the NetDB. vol. 11, pp. 1–7. Athens, Greece (2011)
30. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with pagedattention. In: Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (2023)
31. Legendre, P.: Spatial autocorrelation: trouble or new paradigm? Ecology **74**(6), 1659–1673 (1993)
32. Leverich, J., Kozyrakis, C.: Reconciling high server utilization and sub-millisecond quality-of-service. EuroSys '14, Association for Computing Machinery, New York, NY, USA (2014). https://doi.org/10.1145/2592798.2592821, https://doi.org/10.1145/2592798.2592821
33. Lo, D., Cheng, L., Govindaraju, R., Ranganathan, P., Kozyrakis, C.: Heracles: Improving resource efficiency at scale. In: Proceedings of the 42nd Annual International Symposium on Computer Architecture. pp. 450–462 (2015)
34. Nathan Reed, Nigel Atkinson, V.F.D.T.: Luabridge. https://github.com/vinniefalco/LuaBridge (2007)
35. Reiss, C., Tumanov, A., Ganger, G.R., Katz, R.H., Kozuch, M.A.: Towards understanding heterogeneous clouds at scale: Google trace analysis. Intel Science and Technology Center for Cloud Computing, Tech. Rep **84**, 1–21 (2012)
36. Rose, O.: Estimation of the Hurst parameter of long-range dependent time series, vol. 137. Report (1996)
37. Schroeder, B., Wierman, A., Harchol-Balter, M.: Open versus closed: A cautionary tale (2006)
38. Shumway, R.H., Stoffer, D.S., Shumway, R.H., Stoffer, D.S.: Arima models. Time series analysis and its applications: with R examples pp. 75–163 (2017)
39. Tene, G.: How not to measure latency. Low Latency Summit **68** (2013)
40. Wang, M., Madhyastha, T., Chan, N.H., Papadimitriou, S., Faloutsos, C.: Data mining meets performance evaluation: Fast algorithms for modeling bursty traffic. In: Proceedings 18th International Conference on Data Engineering. pp. 507–516. IEEE (2002)
41. Yin, J., Lu, X., Zhao, X., Chen, H., Liu, X.: Burse: A bursty and self-similar workload generator for cloud computing. IEEE Transactions on Parallel and Distributed Systems **26**(3), 668–680 (2014)